

ASSIGNMENT 6

PROBLEM: program to solve the problem using A* Search in such a way that it can handle the 15-Puzzle problem with the same program.

ALGORITHM:

A* Search for the N-Puzzle Problem:

Step 1: Define load_puzzle(filename)

- This function opens the input file and reads all lines.
- It extracts n , the dimension of the puzzle.
- It constructs the **START** state as a tuple of integers from the next n lines.
- It constructs the **GOAL** state as a tuple of integers from the following n lines.
- Finally, it returns $(n, \text{start_state}, \text{goal_state})$ to be used by the solver.

Step 2: Define manhattan_distance(current_state, goal_state, size)

- This function calculates the **Manhattan distance heuristic**.
- For each tile (ignoring 0), it finds its position in both the current and goal states.
- It computes the row and column differences and sums them across all tiles.
- This heuristic is **admissible** and **consistent**, ensuring the A* algorithm finds the optimal solution.

Step 3: Define generate_neighbors(state, size)

- This function generates all valid next states by sliding the empty tile (0).
- It locates the index of 0 in the state and computes its (row, col) position.
- It considers four possible moves: **up, down, left, right**.
- For each legal move (within puzzle bounds), it swaps 0 with the adjacent tile to form a new state.
- All valid neighbors are returned as a **list of tuples**.

Step 4: Define solve_puzzle(size, start_state, goal_state, intermediate_file)

- A **priority queue** is initialized with the start state.
- The priority of each state is computed as:
- $f(n) = g(n) + h(n)$

- $g(n)$ = cost from start to the current state.
 - $h(n)$ = Manhattan distance heuristic.
- Algorithm process:
 - Pop the state with the lowest $f(n)$ from the queue.
 - Record the state in the explored sequence.
 - If it matches the goal, reconstruct the path and save intermediate steps.
 - Otherwise, generate all valid neighbors:
 - Compute new costs.
 - Update the parent relation.
 - Push neighbors into the priority queue.
- If no solution exists, return None.

Step 5: Define `build_solution_path(parents, current)`

- This function reconstructs the solution path from the goal to the start.
- It backtracks using the `parents` dictionary until the start state is reached.
- The path is then reversed and returned.

Step 6: Run the Solver

- The input file is set as **input.txt**, the solution file as **solution.txt**, and the intermediate output file as **intermediate.txt**.
- Call `load_puzzle()` to get the puzzle size, start state, and goal state.
- Run the solver with `solve_puzzle()`.

Step 7: Output Handling

- If a solution is found:
 - The program prints:
 - Solution found! Path length: <number_of_moves>
 - The full solution path is written to **solution.txt**.
 - All explored intermediate states are written to **intermediate.txt**.
- If no solution exists:
 - The program prints:

No solution exists.

INPUT :

4
1 2 3 4
5 6 0 8
9 10 7 11
13 14 15 12
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

OUTPUT:

1 2 3 4
5 6 0 8
9 10 7 11
13 14 15 12

1 2 3 4
5 6 7 8
9 10 0 11
13 14 15 12

1 2 3 4
5 6 7 8
9 10 11 0
13 14 15 12

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0